Unidad 2 / Escenario 3

Lectura fundamental

Funciones o métodos

Contenido

1 Funciones 1

2 Argumentos y Parámetros 2

3 Métodos y cadenas 3

4 Recursión 6

5 Problemas propuestos 8

Palabras clave: ciclos, funciones, métodos, cadenas y recursión.

En ocasiones, durante procesos de análisis, diseño e implementación de soluciones tecnológicas surgen subproblemas (subrutinas) que se pueden resolver independientemente del problema global que se esté trabajando. Una muy buena práctica de programación es solucionar los problemas con un enfoque modular, en el que se hacen subprogramas con entradas y salidas específicas para estos. Dicha práctica contribuye con varios aspectos, como la legibilidad del código, su reusabilidad, la facilidad en su mantenimiento y documentación, entre otros. Una de las maneras más sencillas y comúnmente usadas en muchos lenguajes de programación es el diseño e implementación de funciones, también conocidas como métodos en Java.

1. Funciones

En lenguajes de programación de propósito general, como Java, la forma más común de hacer fragmentos de código que resuelvan subproblemas y se puedan reutilizar (llamar o invocar) múltiples veces, son los métodos. Algunos ya están definidos e implementados y son procedimientos tan comúnmente usados, que muchas veces se inicia su uso en el aprendizaje de la programación antes de saber formalmente qué son y cómo implementarlos. Ejemplos de ellos son los métodos estáticos en la clase *Math* (Oracle, 1995c), *String* (Oracle, 1995f) y las clases envolventes como *Integer* (Oracle, 1995b) o *Double* (Oracle, 1995a). El proceso de llamar un método en Java se denomina invocación del método. En este lenguaje, los métodos estáticos se deben hacer escribiendo el nombre de la clase, seguido de punto, seguido del nombre del método y, entre paréntesis, los parámetros que recibe (valores con los que se trabajará). Cuando se diseñan métodos propios y estos son estáticos, se pueden invocar dentro de la misma clase sin necesidad de usar el nombre de la clase. Algunos ejemplos de métodos estándar comunes son:

- *Math.max(a,b):* retorna el máximo de los dos argumentos que recibe. a y b pueden ser de varios tipos nativos de datos gracias a la sobrecarga de métodos posible en Java (Oracle,1995d). Esta sobrecarga es básicamente la posibilidad que se tiene en ese lenguaje de programación (y en otros) de definir métodos con el mismo nombre, pero con diferente combinación de parámetros.
- *Math.sin(x)*: retorna el seno (función trigonométrica) del número recibido.
- *Math.sqrt(x)*: retorna la raíz cuadrada del número recibido.
- *Double.compare(a,b)*: sirve para comparar dos números reales. Retorna un valor negativo si a < b; un valor positivo si a > b; y 0 si a y b son numéricamente iguales.
- *Integer.valueOf(a)*: retorna un *Integer* con el valor que representa el argumento a, el cual puede ser un entero o una cadena, gracias a la sobrecarga de métodos que tiene Java.
- *String.valueOf(a)*: retorna una cadena con la representación del valor a, que puede ser cualquier tipo de dato nativo gracias a la sobrecarga de métodos en Java.

Aunque los métodos mencionados y equivalentes en otros lenguajes de programación son estándar, en los lenguajes de programación de alto nivel estos no resuelven todos los problemas posibles a los que se puede enfrentar un programador. Por ello, los lenguajes de programación le permiten a los desarrolladores crear sus propios métodos para tareas específicas. En general, los métodos constan de varias partes estándar en prácticamente todo lenguaje, y de otras opcionales dependiendo de este. Las partes fundamentales de los métodos son su nombre, tipo de dato de retorno, si lo tiene (o *void* si no), y la lista de parámetros.

2. Argumentos y Parámetros

Dependiendo del lenguaje de programación, estos dos términos pueden tener diferentes significados, aunque no es raro ver que se usen como sinónimos. Para diferenciarlos, se puede pensar en el parámetro como la variable encontrada en la definición del método o función, mientras que el argumento es el valor que llega, que se envía a esa función. El parámetro se puede conocer entonces como parámetro o parámetros formales, y el argumento o los argumentos como parámetros reales.

Como primer ejemplo de método (función, subrutina o subprograma) diseñado e implementado en Java, está la tarea de seleccionar y retornar el mayor valor de cuatro posibles valores enteros. Esta puede ser una tarea recurrente en algún programa que se esté diseñando e implementando, como lo muestra el Código 1. Para propósitos de este módulo, se asume que todos los métodos que se hagan serán públicos y estáticos; es decir, llevarán en su definición *public static*. Estas dos palabras reservadas tienen gran importancia en el estudio de la programación orientada por objetos y aunque su análisis detallado se escapa del alcance de este módulo, en este todos los métodos tendrán esas dos palabras justo antes del tipo de dato de retorno, que en este ejemplo es un entero *int*.

En el Código 1, debajo del *main* (que también es un método) se ve la sentencia **public static int max(int a , int b ,int c , int d)**. Esta es la declaración del método y de ella se puede inferir que el nombre del método es *max* y es la forma de invocarlo (llamarlo/usarlo) en el método *main*. En esta definición se puede observar también que sus parámetros son 4 números enteros, los cuales, en el ámbito del método, están en las variables a, b, c y d. El ámbito de la variable es entonces el espacio del código del programa en donde la variable tiene valor, es decir, donde existe y se puede usar. En general, las variables declaradas dentro de los métodos tienen alcance solo en el método dentro del cual están declaradas. Aunque en Java no existe estrictamente la posibilidad de declarar variables globales (asequibles desde toda la aplicación), se puede simular, creándolas pu´blicas y estáticas fuera de todos los métodos. El valor que retorna o que devuelve el método es el que tiene la variable o expresión que está justo después de la instrucción return. Un método puede tener varias instrucciones *return*, pero se debe garantizar que por cualquier flujo de ejecución del programa tendrán una instrucción de retorno. Esto aplica para todos los métodos que no son de tipo *void*, es decir, que no tienen la palabra *void* en la declaración, puesto que esto indicaría que no retornan nada.

```
import java.util.Scanner;
public class Methods {
    public static void main(String[] args) {
        Scanner sn = new Scanner( System.in );
        System.out.print("Type 4 integers: "); int x
        = sn.nextInt();
        int y = sn.nextInt(); int
        z = sn.nextInt(); int w =
        sn.nextInt(); int m =
        max(x,y,z,w);
        System.out.println("The maximum value is: "+m);
    }
    public static int max( int a , int b ,int c , int d )
    {
        if( a > b && a > c && a > d )
            return a;
        if( b > a && b > c && b >
        d )
            return b;
        if( c > a && c > b && c > d )
            return c;
        return d;
    }
}
```

Código 1: primer método en Java. Máximo de 4 enteros

3. Métodos y cadenas

En el Código 2 se muestra un programa, el cual, dada una línea de texto leída de la entrada estándar usando el método *nextLine* de *Scanner*, busca la cantidad de ocurrencias del carácter a en ella. Para ello se diseñó y se muestra un método llamado *count*, que tiene como parámetros una cadena y un carácter. Como resultado de operación del método se retorna la cantidad de ocurrencias del carácter dado en la cadena. Para ello se usan los métodos *length* y *charAt* de la clase *String*, los cuales retornan el tamaño de la cadena y el carácter en una posición determinada de la cadena respectivamente.

```
import java.util.Scanner; public class
StringsAndMethods {
    public static void main(String[] args) {
        Scanner sn = new Scanner( System.in );
        System.out.print("Type a Line: ");
        String line = sn.nextLine();
        char characterToSearch = 'a';
        int occurences = count(line,characterToSearch);
        System.out.println("There are "+occurences+
                " of character "+characterToSearch+ "in
                the String \""+line+"\"");
    }
    private static int count(String s, char c) {
        int answer = 0;
        for (int i = 0; i < s.length(); i++)
            if( c == s.charAt(i))
                ++answer;
            return answer;
    }
}
```

Código 2: Métodos y uso de cadenas.

En el Código 3 se muestra otro programa con ejemplo de uso de varios métodos estándar de Java, unos de la clase *String* y otro de la clase *Character*. Este programa toma como entrada una cadena de texto y como salida muestra la cadena de texto formateada, de modo que la letra inicial de cada palabra sea mayúscula y el resto sean letras minúsculas. Adicional a ello, este programa también elimina posibles redundancias de espacios, haciendo que entre cada par de palabras en la salida haya exactamente un espacio, y que no haya espacios ni tabulaciones al inicio o al final de lo mostrado como salida. Ejemplo: si recibe como entrada la frase Me encanta IA prOgraMación POrqUe es GRATIFICANTE", el programa mostrará como salida: Me Encanta La Programación Porque Es Gratificante. Los métodos usados son:

- *nextLine()*: de la clase *Scanner*. Este método lee de la entrada estándar una cadena hasta que en la entrada haya un salto de línea; es decir, en un IDE. En una consola de comandos sería hasta que encuentre la digitación de un *enter*.
- *toUpperCaseEachFirstLetter()*: es el método diseñado e implementado para hacer la tarea requerida descrita en el párrafo anterior.
- *toLowerCase()*: método de la clase *String* que retorna la cadena usada para invocar el método, pero transformada, de modo que todos los caracteres que son letras estén en minúsculas. Así, por ejemplo, si recibe la frase CadeNa en MulTIcaso, retornará: cadena en multicaso.
- *trim()*: método de la clase *String* que retorna la cadena que se usa para invocarlo, pero con espacios, tabulaciones y saltos de línea eliminados del inicio y final de la cadena.

- *split()*: método de la clase *String* que retorna un arreglo de cadenas (varias cadenas) que se extraen de la cadena que invoca al método, teniendo en cuenta el separador que recibe como argumento. En este caso se usó como separador lo que se puede considerar cualquier ocurrencia de un espacio o tabulación, o combinaciones de ambos [para más detalles sobre su funcionamiento, revisar Oracle (1995e)]. En el Escenario 4 se mostrarán ejemplos particulares de uso de arreglos. Para este caso se usó una estructura *foreach* (for(String str: arr)) que sirve para recorrer todas las palabras separadas según el separador mencionado.
- *Character.toLowerCase()*: método de la clase *Character* que recibe un carácter que, si es letra, lo retorna en mayúscula; en otro caso, retorna el mismo carácter recibido como argumento. Este método se usa para transformar la primera letra de cada palabra a mayúscula; por ello se usa el método *charAt* con el argumento 0.
- append(): método de la clase *StringBuilder* que sirve para agregar contenido a la cadena que se está construyendo con la instancia de *StringBuilder*. Vale la pena anotar que este proceso se podría haber hecho con una variable tipo *String* e ir construyendo con concatenaciones la cadena a retornar. En general, es una buena práctica de programación usar la clase *StringBuilder* en lugar de la clase *String* cuando se deseen hacer procesos en los que muchas concatenaciones pueden ser posibles; esto, por motivos de rendimiento y eficiencia.
- substring(): este método de la clase String es usado para extraer una subcadena a partir de la posición uno de cada cadena después de la separación. Esto se hace para concatenar lo restante de cada cadena, es decir, todo lo que al final quedará en minúscula; todo, exceptuando el primer carácter de cada cadena.
- toString(): método de la clase *StringBuilder* que retorna la representación en *String* de la instancia de *StringBuilder* que invoca el método.

```
import java.util.Scanner; public class
StringAndMethods
{
    public static void main(String[] args)
    {
        Scanner sn = new Scanner( System.in );
        System.out.print("Type a Line: ");
        String line = sn.nextLine();
        String line2 = toUpperCaseEachFirstLetter(line);
        System.out.println(line2);
    }
    public static String toUpperCaseEachFirstLetter( String s )
    {   StringBuilder sb = new StringBuilder();
        String arr[] = s.toLowerCase().trim().split("[ \t]+"); for( String str:
        arr )
        {

            sb.append( Character.toUpperCase( str.charAt(0) ) );
            sb.append( str.substring(1));
            sb.append(' ');
        }
        return sb.toString().trim();
    }
}
```

Código 3: métodos y uso de cadenas

4. Recursión

En computación y en matemáticas hay muchas fórmulas y aproximaciones a problemas que tienen naturaleza recursiva; es decir, se pueden formular como una función definida en términos de ella misma. Dentro de las más conocidas está la serie de Fibonacci, el factorial y los números combinatorios. Otras no tan familiares son los números de Bell, los números de Stirling y diversas ecuaciones en recurrencia que expresan la complejidad de los algoritmos. Para más detalles en esta dirección, revise el capítulo uno de Guerequeta y Vallecillo (2000) y los capítulos 3 y 4 de Cormen, Leiserson, Rivest y Stein (2009)

Como ejemplo de recursión, está la función del combinatorio c(n, k), la cual sirve para calcular la cantidad de maneras de escoger k elementos de un conjunto de n disponibles. Ejemplo, si tenemos los elementos A = {a, b, c, d}, la cantidad de formas de escoger 2 de estos elementos sería c(4, 2) = 6. Un problema relacionado fue el resuelto en la unidad 1, que mostraba cuáles son esas 6 formas; es decir, los subconjuntos de A que tienen taman˜o 2. La fórmula recursiva del combinatorio es la siguiente, la cual se puede encontrar de dos maneras, según la fuente de información consultada:

$$c(n,k) = \begin{cases} 0 & \text{si } k > n \\ 1 & \text{si } \mathrm{k = 0 \text{ ó } n = k} \\ c(n-1,k-1) + c(n-1,k) & \text{en otro caso.} \end{cases}$$

$$\binom{n}{k} = \begin{cases} 0 & \text{si } k > n \\ 1 & \text{si } \mathrm{k = 0 \text{ ó } n = k} \\ \binom{n-1}{k-1} + \binom{n}{k-1} & \text{en otro caso.} \end{cases}$$

Como se puede ver, la función del combinatorio, nombrada por simplicidad como c, está definida en términos de ella misma, puesto que en el tercer caso, que es el general, para determinar el valor de c(n, k) hay que definir los valores c(n − 1, k − 1) y c(n − 1, k). Los dos primeros son los casos base, los cuales se recomienda que siempre vayan resueltos al comienzo, en el código escrito. El primer caso base es equivalente a responder el interrogante cuándo k es mayor que n; es decir, cuando se está preguntando, por ejemplo, de cuántas maneras se pueden escoger 3 elementos de un conjunto de 2 disponibles. El segundo caso base responde a las preguntas: ¿De cuántas maneras puedo escoger 0 elementos (k = 0)? y ¿de cuántas formas puedo escoger todos los elementos (n = k)? La respuesta es 1 en ambos casos.

En el Código 4 se presenta un programa que calcula y muestra el triángulo de Pascal en sus primeras 15 filas. El programa en el método *main* tiene los dos ciclos anidados que sirven para calcular las combinaciones de valores positivos para la función combinatorio, empezando con c(0, 0). El programa almacena cada fila en una variable *StringBuilder*, para después calcular su tamaño y así saber cuántos espacios mostrar antes de presentar la fila de números, de modo que el triángulo de Pascal quede aproximadamente centrado en una pantalla de 80 espacios de ancho.

```
public class Recursion {
    private static final int MAX = 15; private
    static final int MAX_WIDTH = 80; public
    static void main(String[] args)
    {
        for( int i = 0; i < MAX; ++ i )
        {
            StringBuilder sb = new StringBuilder();
            for( int j = 0; j <= i; ++j )
                sb.append(c(i,j)+" ");
            System.out.println(getSpaces((MAX_WIDTH-sb.length())/2)+sb.toString()); }
    }
    public static StringBuilder getSpaces( int n )
    {
        StringBuilder sb = new StringBuilder(); for
        (int i = 0; i < n; i++)
            sb.append(' ');
        return sb;
    }
    public static int c( int n , int k )
    {
        if( k > n )
                return 0;
            if( k == n || k == 0 )
            return 1;
        return c(n-1,k-1) + c(n-1,k);
    }
}
```

Código 4: ejemplo de uso de una función recursiva. El combinatorio

La ejecución del programa 4 debe mostrar la siguiente salida (verificar):

```
                                1
                               1 1
                              1 2 1
                             1 3 3 1
                            1 4 6 4 1
                           1 5 10 10 5 1
                          1 6 15 20 15 6 1
                         1 7 21 35 35 21 7 1
                        1 8 28 56 70 56 28 8 1
                       1 9 36 84 126 126 84 36 9 1
                     1 10 45 120 210 252 210 120 45 10 1
                    1 11 55 165 330 462 462 330 165 55 11 1
                   1 12 66 220 495 792 924 792 495 220 66 12 1
                 1 13 78 286 715 1287 1716 1716 1287 715 286 78 13 1
               1 14 91 364 1001 2002 3003 3432 3003 2002 1001 364 91 14 1
             1 15 105 455 1365 3003 5005 6435 6435 5005 3003 1365 455 105 15 1
           1 16 120 560 1820 4368 8008 11440 12870 11440 8008 4368 1820 560 120 16 1
         1 17 136 680 2380 6188 12376 19448 24310 24310 19448 12376 6188 2380 680 136 17 1
```

Código 5: salida del programa que prueba la función del combinatorio.

5. Problemas propuestos

Casi todos los problemas de esta sección constan de un enunciado en el que se solicita el diseño e implementación de un programa en Java que tome como entrada ciertos datos leídos desde la entrada estándar de ese lenguaje y produzca como única salida los datos solicitados, que pueden ser cálculos o transformaciones hechas con los datos de entrada. En todos los problemas propuestos se deben asumir que las entradas cumplirán con lo estipulado en la sección "`Descripción de entrada`" de cada problema. Dicho esto, se debe entonces diseñar e implementar un programa que cumpla con lo estipulado en la sección "`Descripción de salida`", y que sea únicamente lo solicitado allí, nada más. Teniendo en cuenta que en este documento se estudian las funciones o métodos en Java, se desea que en todos los programas a diseñar e implementar se tengan métodos que hagan tareas específicas. Como requisito adicional recomendado, toda la lectura de datos (uso de *Scanner*) y toda la impresión de datos a la salida estándar (uso de *System.out*) debe hacerse en el método *main*; es decir, en los demás métodos no puede haber ni lectura ni impresión de datos.

1. Número de Fibonacci. Para este programa se recomienda implementar la función recursiva:

$$f(n) = \begin{cases} 0 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ f(n-1) + f(n-2) & \text{en otro caso.} \end{cases}$$

   - Descripción de entrada: un número entero positivo entre 0 y 20.
   - Descripción de salida: un entero positivo, el enésimo número de Fibonacci (The OEIS Foundation, 2018b).

- Ejemplo de entrada:
  0
- Ejemplo de salida:
  1
- Ejemplo de entrada 2:
  1
- Ejemplo de salida 2:
  1
- Ejemplo de entrada 3:
  2
- Ejemplo de salida 3:
  2
- Ejemplo de entrada 4:
  40
- Ejemplo de salida 4:
  102334155

2. Números de Fibonacci en intervalo. En este problema, dados dos números enteros no negativos entre 0 y 10946, se debe retornar la cantidad de números de Fibonacci que están en ese intervalo.
   - Descripción de entrada: dos números enteros entre 0 y 10946 inclusive.
   - Descripción de salida: un entero positivo; la cantidad de números de Fibonacci que hay en ese intervalo (The OEIS Foundation, 2018b).
   - Ejemplo de entrada:
     0 0
   - Ejemplo de salida:
     1
   - Ejemplo de entrada 2:
     4 10
   - Ejemplo de salida 2:
     2
   - Ejemplo de entrada 3:
     1 10
   - Ejemplo de salida 3:
     0
   - Ejemplo de entrada 4:
     0 102334155
   - Ejemplo de salida 4:
     41
3. Número primo. Dado un número entero positivo, decidir si es primo o no.
   - Descripción de entrada: un número entero positivo entre 2 y 100000000.
   - Descripción de salida: *prime* o *not prime*, según el caso.
   - Ejemplo de entrada:
   - 2
   - Ejemplo de salida:
     Prime
   - Ejemplo de entrada 2:
     100000000

- Ejemplo de salida 2:
  Not Prime
- Ejemplo de entrada 3:
  999939
- Ejemplo de salida 3:
  Not Prime
- Ejemplo de entrada 4:
  999983
- Ejemplo de salida 4:
  Prime

4. Números primos en intervalo. En este problema, dados dos números enteros no negativos entre 2 y 10000, se debe retornar la cantidad de números de primos que están en ese intervalo.
   - Descripción de entrada: dos números enteros entre 2 y 10000 inclusive.
   - Descripción de salida: un entero positivo; la cantidad de números primos que hay en ese intervalo.
   - Ejemplo de entrada:
     2 2
   - Ejemplo de salida:
     1
   - Ejemplo de entrada 2:
     2 11
   - Ejemplo de salida 2:
     5
   - Ejemplo de entrada 3:
     2 10000
   - Ejemplo de salida 3:
     1220
   - Ejemplo de entrada 4:
     9967 9973
   - Ejemplo de salida 4:
     2
5. Número de Bell. Para este programa se recomienda implementar la función recursiva:

$$B(n) = \begin{cases} 1 & \text{si } n = 0 \\ 1 & \text{si } n = 1 \\ 2 & \text{si } n = 2 \\ \sum_{k=0} \binom{}{k} B(k) & \text{en otro caso.} \end{cases}$$

   - Descripción de entrada: un nu´mero entero positivo entre 0 y 20.
   - Descripción de salida: un entero positivo el enésimo número de Bell (The OEIS Foundation, 2018a).
   - Ejemplo de entrada:
     0
   - Ejemplo de salida:
     1
   - Ejemplo de entrada 2:
     1

- Ejemplo de salida 2:
  1
- Ejemplo de entrada 3:
  10
- Ejemplo de salida 3:
  115975
- Ejemplo de entrada 4:
  20
- Ejemplo de salida 4:
  51724158235372

6. Números de Bell en intervalo. En este problema, dados dos números enteros no negativos entre 0 y 51724158235372, se debe retornar la cantidad de números de Bell que están en ese intervalo.
   - Descripción de entrada: dos números enteros entre 0 y 51724158235372 inclusive.
   - Descripción de salida: un entero positivo; la cantidad de números de Bell que hay en ese intervalo (Foundation, 2018b).
   - Ejemplo de entrada:
   - 1 1
   - Ejemplo de salida:
     1
   - Ejemplo de entrada 2:
     1 10
   - Ejemplo de salida 2:
     4
   - Ejemplo de entrada 3:
     53 3139
     Ejemplo de salida 3:
   - 2
     Ejemplo de entrada 4:
   - 0 51724158235372
     Ejemplo de salida 4:
   - 21
7. Función recursiva. Diseñar e implementar un programa en java que tenga un método que implemente la siguiente función recursiva:

$$f(n) = \begin{cases} 0 & \text{si } n = 0 \\ (-n)! & \text{si } n \text{ es negativo.} \\ n! * (n-1)! & \text{si } n \text{ es impar y positivo.} \\ 2^n * (n/2)! & \text{si } n \text{ es par y positivo.} \end{cases}$$

   - Descripción de entrada: dos números enteros entre -15 y 15 inclusive.
   - Descripción de salida: un entero positivo; el valor de la función dada.
   - Ejemplo de entrada:
     0
   - Ejemplo de salida:
     0
   - Ejemplo de entrada 3:
     3

- Ejemplo de salida 3:
  12
- Ejemplo de entrada 4:
  -2
- Ejemplo de salida 4:
  2

8. Hacer un seguimiento paso a paso al siguiente programa, asumiendo que se lee el código estudiantil o número de cédula. Recomendación: usar una hoja de cálculo o papel y lápiz. Después, correr el programa con la entrada que se hizo el seguimiento y verificar que todas las salidas correspondan a las detectadas durante este.

```
import java.util.Scanner;
public class Main1 {
    public static void main(String[] args) {
        Scanner sn = new Scanner( System.in );
        System.out.println(mistery(sn.nextInt()));
        sn.close();
    }
    public static int mistery( int n ){
        int x,q,d=6;
        q = (int)Math.pow(10,d);
        x = n%q;
    return misteryAux(x,d);
    }
    public static int misteryAux( int n , int d )
    {
        if( d == 1 )
        {
            return n;
        }
        int a,r,p;
        p = (int)Math.pow(10,d); r
        = n%10;
        d = d-1;
        n = n/10;
        a = p*r;
        a = a/10;
        a = a + misteryAux(n,d);
        return a;
    }
}
```

Código 6: seguimiento a programa paso a paso.

9. Invertir número. Diseñar e implementar un programa que tome como entrada un número entero y lo invierta. Implementar el programa sin usar funciones estándar del lenguaje, como el método *reverse* de *StringBuilder*; sin emplear arreglos, cadenas u otro tipo de colecciones. Recomendación: solo utilizar operaciones entre enteros (división parte entera y residuo al dividir en potencias de 10).
   - Descripción de entrada: dos números enteros entre -4611686018427387904 y 4611686018427387904 inclusive. Asumir que el dígito menos significativo (más a la derecha) no será 0.

- Descripción de salida:
  el entero invertido.
- Ejemplo de entrada:
  1
- Ejemplo de salida:
  1
- Ejemplo de entrada 2:
  -1
- Ejemplo de salida 2:
  -1
- Ejemplo de entrada 3:
  -12349
- Ejemplo de salida 3:
  -94321
- Ejemplo de entrada 4:
  999888777999888
- Ejemplo de salida 4:
  888999777888999

10. Diseñar e implementar un programa que calcule la función de Ackerman para dos argumentos no negativos dados. Para este programa se recomienda implementar la función recursiva:

$$A(m,n) = \begin{cases} n+1 & \text{si } m = 0 \\ A(m-1,1) & \text{si } n = 0 \\ A(m-1, A(m,n-1)) & \text{en otro caso} \end{cases}$$

- Descripción de entrada: dos números, ambos entre 0 y 3 inclusive.
- Descripción de salida: un entero positivo. La función de Ackerman evaluada en los números dados (Wolfrand Mathworld, s.f.).
- Ejemplo de entrada:
  0 0
- Ejemplo de salida:
  1
- Ejemplo de entrada 2:
  3 3
- Ejemplo de salida 2:
  61
- Ejemplo de entrada 3:
  3 2
- Ejemplo de salida 3:
  29
- Ejemplo de entrada 4:
  2 3
- Ejemplo de salida 4:
  9

Referencias

Cormen, T., Leiserson, C., Rivest, R. y Stein, C. (2009). *Introduction to Algorithms.* Cambridge, MA, EUA: The MIT Press.

Guerequeta, R. y Vallecillo, A. (2000). *Técnicas de Diseño de Algoritmos.* Málaga, España: Servicio de Publicaciones de la Universidad de Malaga.

Oracle. (1995a). *Class Double*. Recuperado de https://docs.oracle.com/javase/7/docs/api/java/lang/Double.html

Oracle. (1995b). *Class Integer*. Recuperado de https://docs.oracle.com/javase/7/docs/api/java/lang/Integer.html

Oracle. (1995c). *Class Math* . Recuperado de https://docs.oracle.com/javase/7/docs/api/java/lang/Math.html

Oracle. (1995d). *Defining Methods.* Recuperado de https://docs.oracle.com/javase/tutorial/java/javaOO/methods.html

Oracle. (1995e). *Class Pattern.* Recuperado de https://docs.oracle.com/javase/7/docs/api/java/util/regex/Pattern.html

Oracle. (1995f). *Class String Class*. Recuperado de https://docs.oracle.com/javase/7/docs/api/java/lang/String.html

The OEIS Foundation. (2018a). *The On-Line Encyclopedia of Integer Sequences: Bell Numbers*. Recuperado de https://oeis.org/A000110/list

The OEIS Foundation . (2018b). *The On-Line Encyclopedia of Integer Sequences: Fibonacci Numbers.* Recuperado de https://oeis.org/A000045

Wolfrand Mathworld. (s.f.). *Ackerman Function.* Recuperado de http://mathworld.wolfram. com/AckermannFunction.html

INFORMACIÓN TÉCNICA

Ingeniería y Ciencias Básicas

Módulo: Conceptos Fundamentales de Programación
Unidad 2: Funciones y arreglos
Escenario 3: Funciones o métodos

Autor: Rafael Niquefa

Asesor Pedagógico: Angie Laitón
Diseñador Gráfico: Rafael Niquefa
Asistente: Alejandra Morales

Este material pertenece al Politécnico Grancolombiano. Por ende, es de uso exclusivo de las Instituciones adscritas a la Red Ilumno. Prohibida su reproducción total o parcial.